Hand Gesture Classification on Edge Devices

Team 14
王昱閔　王辰竹　羅詠璿

Abstract

We present a compact multimodal hand-gesture classifier for edge deployment. The model receives only a cropped RGB hand image and 21 MediaPipe landmarks, then predicts one of five command gestures or rejects the input as N/A. A depthwise-separable convolutional branch captures appearance, while a landmark multilayer perceptron captures hand geometry. Their features are fused and followed by confidence-based rejection calibrated for an asymmetric evaluation rule in which a false trigger is penalized twice as heavily as a correct command is rewarded. The final network contains 36,039[VERIFY] trainable parameters. Its tensors occupy 0.142 MiB[VERIFY], while the submitted PyTorch checkpoint occupies 0.168 MiB[VERIFY]. The best reported validation accuracy is 96.75%[PROG], and the stronger of two leaderboard submissions obtained a pre-presentation subtotal of 81.4/90[PRES].

1 Introduction

Touch-free gesture interfaces must recognize intended commands while avoiding activation on ordinary hand poses. This requirement is especially important on edge devices, where models must also remain compact and operate without cloud inference. The course challenge fixes the upstream MediaPipe preprocessor and permits the downstream classifier to receive only a cropped RGB hand image and 21 hand landmarks [1, 2].

The task uses five valid commands—fist, like, OK, one, and palm—and maps every other accepted one-hand pose to N/A. Its scoring rule gives one point for a correct target prediction but subtracts two points for a false trigger [1]. This asymmetry changes the design objective: maximizing closed-set accuracy alone is insufficient, because uncertain samples should often be rejected.

Our main contributions are:

- a dual-stream model combining visual appearance and landmark geometry within a small parameter budget;
- a three-stage training procedure consisting of image pretraining, joint multimodal optimization, and scoring-aware calibration; and
- a conservative inference policy combining neural N/A prediction, confidence and margin gates, and landmark validity checking.

2 Task and Evaluation

2.1 Inputs and label space

The deployed function follows the required `predict(cropped_img, landmarks)` interface [INFER]. The image is resized to 64×64[INFER] RGB pixels and normalized channel-wise from $[0, 1]$ to $[-1, 1]$. The 21[SPEC] crop-relative (x, y) landmarks are flattened into a 42[INFER]-dimensional vector. The output mapping is shown in Table 1. Full-frame images are prohibited [1].

2.2 Scoring rule

The official evaluation allocates 30 points to model size, 20 to HaGRIDv2 performance, 40 to real-world robustness, and 30 to presentation [1]. For a model no larger than 10 MB, the size score is

$$S_{\text{size}} = (10 - \text{size}_{\text{MB}}) \times 3. \tag{1}$$

Both performance sets use the raw contribution

$$s_i = \begin{cases} +1, & \text{correct target command,} \\ -2, & \text{false trigger,} \\ 0, & \text{N/A output.} \end{cases} \tag{2}$$

The robustness set contains 50 N/A interference images and 50 target images [1]. Since one false trigger cancels two correct commands, we optimize checkpoint selection and calibration with this scoring function in addition to reporting accuracy.

Table 1: Output label mapping used by the submitted inference code.

ID	Label	ID	Label
0	N/A	3	OK
1	fist	4	one
2	like	5	palm

3 Related Work

MediaPipe Hands combines a palm detector and a landmark estimator for real-time on-device hand tracking [2]. Its landmark representation provides geometric information that is less dependent on texture and illumination than RGB appearance alone. HaGRIDv2 is a large-scale gesture dataset containing static, dynamic, and no-gesture examples [3]. Our classifier uses its 512-pixel release while following the challenge-specific target and N/A mapping.

To reduce computation, the image encoder uses depthwise-separable convolutions, a factorization used by MobileNets to replace a standard spatial convolution with depthwise and pointwise operations [4]. The proposed network is task-specific and trained from scratch; it does not use a gesture-recognition pretrained model.

4 Proposed Method

4.1 Dual-stream architecture

Figure 1 summarizes the model. The RGB crop and landmark vector are encoded independently, producing two 64-dimensional descriptors. These descriptors are concatenated into a 128-dimensional multimodal feature, which is classified into six logits. The image branch contains 13,249[VERIFY] trainable parameters, the landmark branch contains 14,144[VERIFY], and the fusion head contains 8,646[VERIFY]. This division gives both modalities comparable model capacity while keeping the complete network small.

4.2 Image branch

The image branch contains five depthwise-separable blocks. For an input with C channels, the 3×3 depthwise convolution uses C groups and filters each channel independently. A 1×1 pointwise convolution then mixes information across channels and changes the channel count. Each operation is followed by batch normalization and ReLU [5]. This factorization replaces a dense spatial convolution while preserving its local receptive field.

Table 2 gives the complete tensor progression. The first three blocks use stride two and reduce the spatial resolution from 64×64 to 8×8[ARCH]; Blocks 4 and 5 refine the features without further downsampling. Adaptive global average pooling averages each of the final 64 feature maps over its 8×8 spatial grid, yielding one scalar per channel and therefore a 64-dimensional[ARCH] image descriptor.

Table 2: Detailed image-branch architecture. Every block follows DWConv–BN–ReLU–PWConv–BN–ReLU.

Layer	Input tensor	Output tensor	Stride
Block 1	64×64×3	32×32×16	2
Block 2	32×32×16	16×16×32	2
Block 3	16×16×32	8×8×64	2
Block 4	8×8×64	8×8×64	1
Block 5	8×8×64	8×8×64	1
GAP	8×8×64	64	–

4.3 Landmark branch and fusion

The landmark input preserves the order of the 21 MediaPipe points. Its crop-relative (x, y) coordinates are flattened in landmark order, producing 42 values. The landmark branch first projects 42→128[ARCH], applies batch normalization and ReLU, then projects 128→64[ARCH] followed by another normalization and ReLU. Unlike the image branch, this branch operates directly on finger and palm geometry and is unaffected by crop texture or color.

Let $\mathbf{f}_I \in \mathbb{R}^{64}$ and $\mathbf{f}_L \in \mathbb{R}^{64}$ denote the image and landmark descriptors. Fusion is performed by direct concatenation,

$$\mathbf{f} = [\mathbf{f}_I; \mathbf{f}_L] \in \mathbb{R}^{128}. \tag{3}$$

The fusion head applies a 128→64[ARCH] linear layer and ReLU, then 0.3[ARCH] dropout during training. A final 64→6[ARCH] layer produces one logit for N/A and one for each target gesture. Softmax is applied only during evaluation and inference to convert these logits into class probabilities.

4.4 Conservative N/A rejection

The deployed decision procedure contains three safeguards [INFER]. Before running the network, it computes the horizontal and vertical span of the 21 crop-relative landmarks. If the larger span is below 0.05[INFER], all points have collapsed into a very small region and the sample is immediately rejected as N/A. For a valid sample, the image and landmarks pass through the two branches and softmax produces six probabilities. If the highest-probability class is already class 0, the model returns N/A directly. Otherwise, let p_1 be the highest non-N/A prediction and p_2 the runner-up probability. The sample is rejected when

$$p_1 < \tau_c \quad \text{or} \quad p_1 - p_2 < \tau_m. \tag{4}$$

Calibration selected $\tau_c = 0.95$[CAL] and $\tau_m = 0.00$[CAL]. Therefore, the final deployed sequence is: landmark validity check, six-class neural prediction, direct neural N/A decision, and finally the 0.95 confidence gate. The margin condition remains implemented for reproducibility and future recalibration, but a zero threshold means it does not reject additional samples in the submitted configuration.

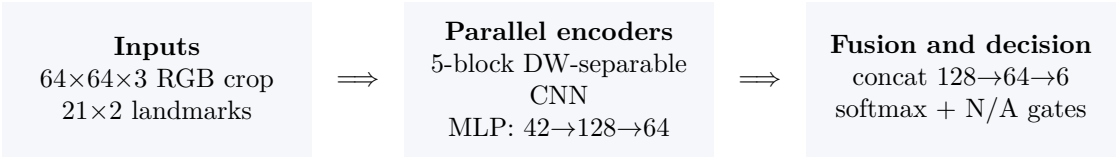

Figure 1: Overview of the dual-stream classifier. The image and landmark encoders each produce a 64-dimensional feature before concatenation.

5 Dataset and Preprocessing

5.1 Dataset construction

We use the HaGRIDv2 512-pixel dataset [3]. The source dataset IDs 2, 9, 14, 15, and 16 are mapped respectively to fist, like, OK, one, and palm, which become output classes 1–5. Every retained one-handed category outside this set is assigned class 0 (N/A). Source IDs {5,6,7,8,23,30,33}[DATA], which represent two-handed categories, are excluded rather than converted to N/A because the challenge classifier receives a single detected hand. This policy makes the N/A class a broad collection of non-command one-hand poses instead of a single semantic gesture. The exact mapping constants are retained in the submitted preprocessing source code.

5.2 MediaPipe crop generation

The preprocessor detects at most one[DATA] hand using detection, presence, and tracking confidence thresholds of 0.5[DATA]. Given the 21 image-relative landmarks, it finds their minimum and maximum image coordinates to form a tight bounding box. Padding equal to 30%[DATA] of the larger box dimension is added on every side, and the padded box is clamped to the image boundary. The RGB pixels inside this box form the classifier crop.

Landmarks are then converted from full-image coordinates to crop-relative coordinates. For a crop (l, t, r, b) in an image of width W and height H, the transformation is

$$x_c = \frac{xW - l}{r - l}, \qquad y_c = \frac{yH - t}{b - t}. \tag{5}$$

Images for which MediaPipe detects no hand are discarded. For each accepted sample, the pipeline stores the crop as JPEG and the 21×2 landmark array as a NumPy file; the original full-frame image is not retained.

Table 3 shows a substantial imbalance: the N/A split is approximately eight times the size of an individual target class [PRES]. Phase 1 therefore uses inverse-frequency sampling, while Phase 2 uses inverse-frequency loss weights.

5.3 Landmark-synchronized augmentation

The official test applies reasonable perturbations such as bounding-box jitter and blur [1]. Training therefore applies ±10%[AUG] crop jitter, Gaussian blur with $\sigma \in [0.1, 1.5]$[AUG], brightness/contrast/saturation jitter of ±20%[AUG], horizontal flip with probability 0.5[AUG], and rotation up to ±15°[AUG]. Spatial changes update the landmarks together with the image. For example, rotation around the crop center uses

$$x' = (x - .5)\cos\theta - (y - .5)\sin\theta + .5, \tag{6}$$
$$y' = (x - .5)\sin\theta + (y - .5)\cos\theta + .5. \tag{7}$$

This prevents image-landmark disagreement during multimodal training.

Table 3: Processed split distribution reported by the team presentation. Counts reflect samples retained after preprocessing.

Split	N/A	fist	like	OK	one	palm	Total
Train	487094	21079	20421	21646	21267	22196	593703
Val.	58407	2669	2625	2816	2661	2804	71982
Test	99385	4602	4568	4808	4633	4784	122780

Source: Team 14 presentation [PRES]. The original machine-readable count log was not committed.

6 Training and Calibration

6.1 Phase 1: image pretraining

The first stage trains the image branch with a temporary 64→6[P1] head for 20[P1] epochs. We use Adam [6] with learning rate 10^{-3}[P1], batch size 256[P1], cross-entropy loss, and cosine annealing. A `WeightedRandomSampler` assigns each class equal expected sampling weight. The project progress record reports a best validation accuracy of 86.15%[PROG].

6.2 Phase 2: joint optimization

The best Phase-1 image weights initialize the full model. Joint training runs for 30[P2] epochs with batch size 256[P2]. The pretrained image branch uses learning rate 10^{-4}[P2], while the new landmark and fusion layers use 5×10^{-4}[P2]. The loss uses inverse-frequency class weights, with the N/A weight multiplied by 1.5[P2] to encourage conservative decisions. Checkpoints are selected by validation contest score rather than accuracy alone. The reported best validation accuracy is 96.75%[PROG].

6.3 Threshold calibration

After training, softmax outputs for the validation set are cached and scored over a $14 \times 11 = 154$[CAL] grid. Confidence thresholds span 0.30–0.95[CAL]; margin thresholds span 0.00–0.50[CAL]. Figure 2 shows that the best validation contest score, 11,936[HEAT], occurs at the conservative confidence threshold of 0.95.

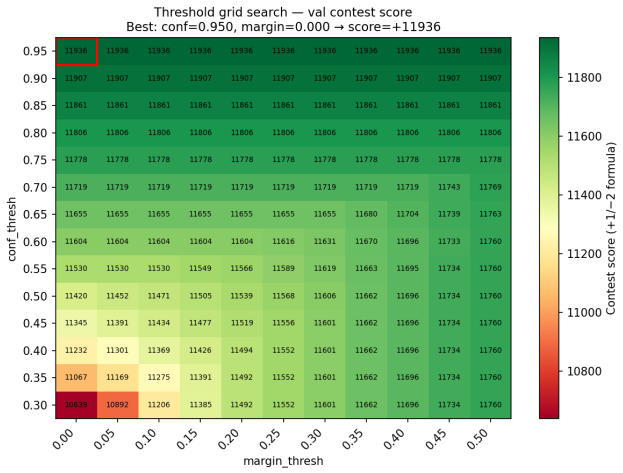

Figure 2: Validation contest score over the calibration grid. Each cell applies one confidence-margin pair to the same cached validation probabilities and evaluates the official $+1/-2$ objective.

7 Results

7.1 Validation and model size

The architecture contains 36,039[VERIFY] trainable parameters. There are two relevant size measurements. Summing model parameters and registered buffers gives 0.142 MiB[VERIFY], the value shown in the architecture slides. The serialized `weights.pt` file is 176,372 bytes (0.168 MiB)[VERIFY] because PyTorch serialization adds metadata. The latter is consistent with the leaderboard model-size score of 29.5/30[PRES]. Distinguishing these quantities resolves the apparent difference between the presentation's "0.142 MB" model statistic and the submitted file size.

7.2 Leaderboard performance

Run 1 achieved the stronger overall subtotal because its 1.5-point robustness advantage outweighed Run 2's 0.6-point basic-performance advantage. Run 2 reached the maximum reported basic-performance score, but the lower robustness score suggests that model selection must account for real-world N/A examples rather than relying only on the HaGRIDv2-derived test set.

Table 4: Training and calibration results reported by project artifacts.

Stage	Validation accuracy	Contest score
Image pretraining	86.15%[PROG]	–
Joint dual-stream	96.75%[PROG]	–
Calibrated model	–	11936[HEAT]

Table 5: Two official-evaluation runs reported in the presentation. "Total" is the subtotal before the 30-point presentation component.

Run	Size	Basic	Robustness	Total / 90
1	29.5	19.4	32.5	**81.4**
2	29.5	**20.0**	31.0	80.5

All values in this table are transcribed from the final presentation[PRES].

8 Discussion and Limitations

The results support three practical conclusions. First, a small custom network is sufficient for a high challenge score: the submission uses less than 0.2 MiB of serialized weights while retaining strong target-set performance. Second, calibration matters because the evaluation heavily penalizes false activation. The selected threshold of 0.95 deliberately trades recall for precision. Third, the gap between basic and robustness scores indicates a remaining domain shift between HaGRIDv2 imagery and TA-shot daily-action examples.

The available evidence also has limitations. The repository contains the training and calibration code, final weights, thresholds, and heatmap, but not the original Phase-1/Phase-2 logs or per-example leaderboard predictions. Consequently, the validation accuracies are traceable to the project progress record, while the leaderboard table is traceable to the team presentation; neither was independently recomputed in this report. The project also did not preserve image-only and landmark-only ablation runs, so the separate contribution of each modality cannot be quantified. Finally, latency and energy consumption were not measured on a specified edge device; small model size alone does not establish real-time performance on every deployment target.

9 Reproducibility and Evidence

Table 6 maps every evidence marker used beside a quantitative claim to source code, a generated artifact, the official specification, or the final presentation. The submitted report folder includes the presentation PDF and a verification script for model statistics.

Table 6: Evidence ledger for implementation details and quantitative claims.

Key	Authoritative source	Claims supported
SPEC	`source/Final_Hand_Gesture_specs.pdf`; Ref. [1]	Inputs, class definitions, limits, scoring rules, and test composition.
INFER	`inference.py`	Input normalization, label mapping, thresholds, and rejection logic.
ARCH	`model/architecture.py`	Layer dimensions, feature dimensions, and dropout.
AUG	`train/augment.py`; `train/dataset.py`	Augmentation types, probabilities, and parameter ranges.
P1	`train/train_phase1.py`	Phase-1 architecture and hyperparameters.
P2	`train/train_phase2.py`	Phase-2 optimization, loss weighting, and checkpoint selection.
CAL	`train/calibrate.py`; `model/thresholds.json`	Grid definition and deployed thresholds.
HEAT	`figures/threshold_heatmap.png`; `train/plot_threshold_heatmap.py`	Grid-search scores and selected calibration point.
VERIFY	`evidence/verify_report_numbers.py`; `model/architecture.py`; `model/weights.pt`	Parameter count, tensor footprint, checkpoint size, and computed size score.
DATA	`download_and_preprocess.py`; `hand_preprocess.py`	Dataset source, label mapping, exclusion policy, and preprocessing.
PROG	`PROGRESS.md`	Recorded Phase-1 and Phase-2 validation accuracies and the selected training run. Original training logs are not present in the repository.
PRES	`source/Hand Gesture Classification on Edge Devices.pdf`; Ref. [7]	Retained split counts and leaderboard results. The original evaluation export or screenshot is not present in the repository.

10 Conclusion

We developed a compact dual-stream hand-gesture classifier that combines RGB appearance with MediaPipe landmark geometry and uses calibrated rejection to address asymmetric false-trigger costs. The final submission contains only 36,039 trainable parameters, and its strongest reported leaderboard run scored 81.4 out of 90 before presentation points. The remaining performance gap is primarily in real-world robustness, motivating future work on hard-negative collection, explicit modality ablation, and evaluation on measured edge hardware.

Code Availability

The complete inference and training source code, final model weights, threshold configuration, and environment requirements are available at https://github.com/xxjustintwxx/Multimedia_Final_Project. The report package includes the exact presentation, specification, figures, and verification helper used to support its claims.

References

[1] Multimedia Course Teaching Team and Microsoft, "Hand gesture classification on edge devices: Challenge specification," 2026, course project specification, local file: Final_Hand_Gesture_specs.pdf.

[2] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "Mediapipe hands: On-device real-time hand tracking," *arXiv preprint arXiv:2006.10214*, 2020. [Online]. Available: https://arxiv.org/abs/2006.10214

[3] A. Nuzhdin, A. Nagaev, A. Sautin, A. Kapitanov, and K. Kvanchiani, "HaGRIDv2: 1m images for static and dynamic hand gesture recognition," *arXiv preprint arXiv:2412.01508*, 2024. [Online]. Available: https://arxiv.org/abs/2412.01508

[4] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017. [Online]. Available: https://arxiv.org/abs/1704.04861

[5] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the International Conference on Machine Learning*, 2015, pp. 448–456. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.html

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[7] Team 14, "Hand gesture classification on edge devices," Jun. 2026, final project presentation, June 11, 2026.

Team Member Contributions

Table 7: Team member contribution statement. Complete the student IDs and concrete deliverables before submission.

Member	Student ID	Specific contributions
王昱閔	[ID]	[Specify data, implementation, experiments, writing, and presentation work.]
王辰竹	[ID]	[Specify data, implementation, experiments, writing, and presentation work.]
羅詠璿	[ID]	[Specify data, implementation, experiments, writing, and presentation work.]